

# Accelerating BIS Standard Discovery for MSEs

Indian Micro and Small Enterprises face significant friction in regulatory compliance due to the complexity of identifying applicable Bureau of Indian Standards. Current manual search through lengthy PDF documents requires domain expertise and can take days to weeks. Our solution transforms this into an automated precision retrieval system that converts natural language product descriptions into relevant BIS standards with high accuracy and minimal latency.



# Deterministic RAG-based BIS Recommendation Engine

## System Overview

We built a Retrieval-Augmented Generation system with a critical design choice: **retrieval is fully deterministic** and the LLM is optional, used only for explanation. This ensures hallucination-free retrieval where standard IDs are retrieved directly from stored data, not generated.

The pipeline extracts structured content from BIS PDFs, converts into standard-centric chunks, generates embeddings using sentence-transformers, and stores vectors in FAISS index for efficient nearest neighbor search.



### Evaluation Mode

Pure retrieval returns top-k standards for batch processing and scoring



### Interactive Mode

Retrieval plus LLM explanation via quantized GGUF model



### Compliance-Critical

Separates truth from generation to ensure reliability

# End-to-End RAG Architecture

Modular architecture with clearly separated responsibilities enables reproducibility, scalability, and debugging clarity.

01

---

## Data Pipeline

`extract_text.py` uses PyMuPDF to extract and clean raw text. `chunker.py` creates structured JSON with standard IDs, content, and categories. `filter_chunks.py` removes low-quality entries.

02

---

## Vector Pipeline

`embedder.py` combines category and content, generates embeddings using BAAI/bge-base-en-v1.5, and stores in FAISS IndexFlatL2 for exact search.

03

---

## Retrieval Layer

`retriever.py` loads models and index, performs query embedding, L2 nearest neighbor search, deduplication, and top-k selection with zero hallucination risk.

04

---

## Generation Layer

`generator.py` loads GGUF model via llama.cpp, formats ChatML prompts, and generates grounded responses when explanation is requested.

05

---

## Interfaces

`inference.py` handles batch evaluation with latency measurement. `app.py` provides Gradio chatbot UI for interactive exploration.

```

    "content": "COARSE AND FINE AGGREGATES",
    "category": "CEMENT AND CONCRETE AGGREGATES",
  },
  {
    "standard": "IS 2116: 1980",
    "content": "SAND FOR MASONRY MORTARS Sc",
    "category": "CEMENT AND CONCRETE AGGREGATES",
  },
  {
    "standard": "IS 9142: 1979",
    "content": "ARTIFICIAL LIGHTWEIGHT AGGR",
    "category": "CEMENT AND CONCRETE AGGREGATES",
  },
  {
    "standard": "IS 269: 1989",
    "content": "ORDINARY PORTLAND CEMENT, 3",
    "category": "CEMENT AND CONCRETE CEMENT",
  },
  {
    "standard": "IS 455: 1989",
    "content": "PORTLAND SLAG CEMENT (Fourth",
    "category": "CEMENT AND CONCRETE CEMENT",
  },
  {
    "standard": "IS 1489 (Part 1): 1991",
    "content": "PORTLAND POZZOLANA CEMENT PA",
    "category": "CEMENT AND CONCRETE CEMENT",
  },

```

# Precision-Oriented Chunking & Deterministic Retrieval

## Chunking Strategy

Chunk unit equals one BIS standard summary extracted from SUMMARY OF blocks. Each chunk contains standard ID, cleaned scope with requirements, and category inferred from document structure.

This approach aligns chunk boundaries with semantic units, prevents context fragmentation, and improves embedding quality through scope extraction, bullet parsing, and revision noise removal.

## Retrieval Strategy

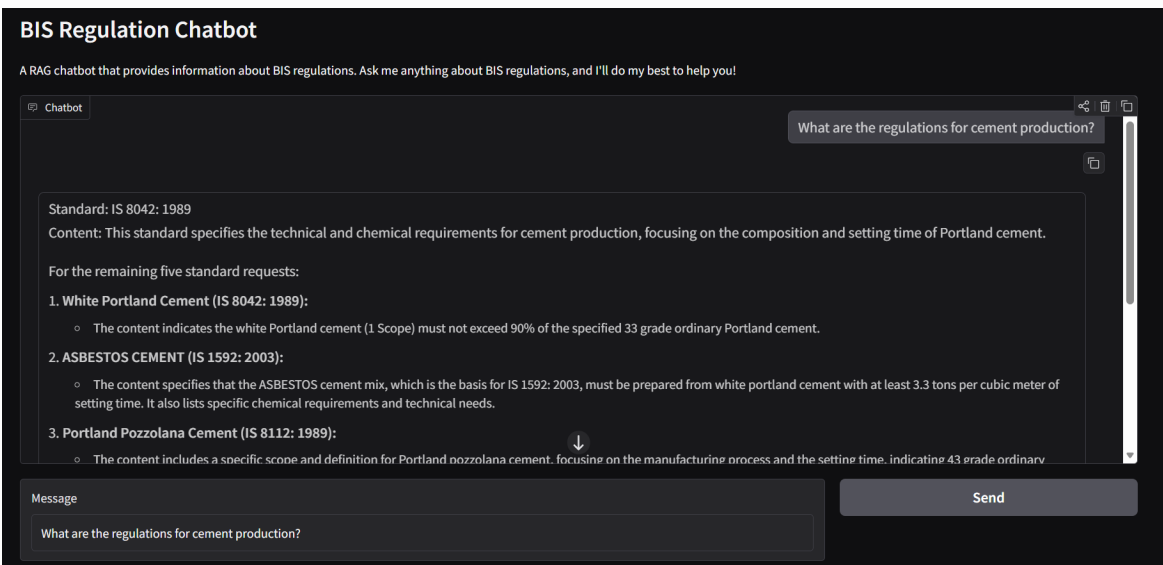
SentenceTransformer generates embeddings from category plus content. FAISS IndexFlatL2 performs exact L2 similarity search with no approximation.

Query flow: lowercase query → generate embedding → retrieve top-k matches → deduplicate by standard ID. No reranking or cross-encoder used.



**Critical Design Choice:** Deterministic retrieval with no LLM involvement in ranking ensures zero hallucination risk while maintaining interpretability.

# Live Demo: Chat Interface & Bulk Inference Pipeline



## Interactive Chatbot Interface

Built using Gradio UI, users enter natural language queries like "cement standards" and receive instant recommendations. System displays retrieved standards with categories and optional LLM-generated explanations. Full transparency shows exactly what was retrieved.

### Real-Time & Batch Modes

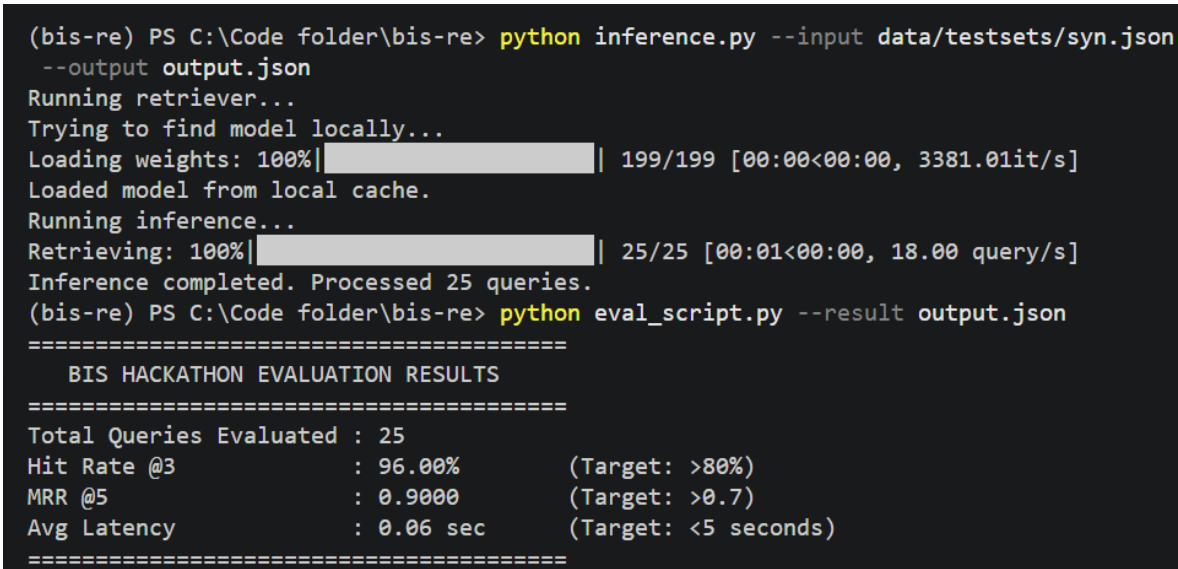
Supports both interactive exploration and large-scale evaluation workflows

### Sub-Second Latency

Average query time of 0.05 seconds provides responsive user experience

### Transparent Results

Explainable retrieval with deterministic, reproducible evaluation pipeline



## CLI-Based Bulk Inference

Executed via inference.py with test.json input and results.json output. Processes entire dataset using deterministic retrieval pipeline with per-query latency measurement. Outputs JSON formatted for evaluation script with real-time progress tracking via tqdm.



# Performance Metrics & Benchmark Results

Metrics computed using official evaluation script exceed hackathon targets by strong margins.

| Testset     | Hit@3 | MRR  | Latency(s) |
|-------------|-------|------|------------|
| Public      | 100%  | 0.95 | 0.06       |
| Synthetic 1 | 96%   | 0.90 | 0.07       |
| Synthetic 2 | 93%   | 0.90 | 0.04       |
| Synthetic 3 | 86%   | 0.81 | 0.04       |
| Average     | 94%   | 0.89 | 0.05       |

## Average Performance

- **Hit@3: 94%** — Exceeds 80% target by 14 points
- **MRR: 0.89** — High ranking quality with correct answers appearing early
- **Latency: 0.05s** — ~100x faster than required 5-second limit

## Why Performance Is Strong

- Exact FAISS search with no approximation loss
- High-quality chunking aligned with standards
- No reranking overhead ensures ultra-low latency
- Deterministic retrieval guarantees consistent accuracy



# Real-World Impact for Small Enterprises



## Before

Manual compliance discovery requiring domain experts takes days or weeks with risk of missing critical standards



## Transformation

Instant recommendations in seconds from simple product descriptions



## After

Relevant BIS standards delivered instantly in structured, explainable format



## Reduced Time and Cost

From days/weeks to seconds



## Increased Accuracy

94% Hit@3 precision



## Democratized Access

Standards for all MSEs

System extends to other domains (electronics, chemicals) and regulatory bodies. Can integrate into ERP systems and compliance platforms, transforming compliance from manual bottleneck into automated intelligence layer.

# Team, Tools & Acknowledgements



**Sarthak Sharan**

Heritage Institute of Technology

## Tech Stack

- Embeddings: SentenceTransformers (BAAI/bge-base-en-v1.5)
- Vector DB: FAISS (IndexFlatL2)
- LLM: llama.cpp (GGUF quantized)

## Acknowledgements

We thank the hackathon organizers, BIS for the SP 21:2005 dataset, and the open-source ecosystem including HuggingFace, FAISS, and llama.cpp that made this project possible.



**Mohar Das**

Indian Institute of Technology, Madras

- UI Framework: Gradio
- PDF Parsing: PyMuPDF
- CLI Interface: argparse + tqdm